Growth of functions

Part I Foundations

Scope and prerequisites

Use full quantifiers, constants, and counterexamples to establish five asymptotic relations. Distinguish input costs from function bounds, then derive conditions for floor and mod, fixed-base growth comparisons, factorials, log-star, and Fibonacci numbers. Prerequisites are real numbers, logarithms, polynomials, and elementary inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- 3.1 Asymptotic notation. Define O, Ω, and Θ with precise quantifiers and prove polynomial leading-term bounds.
- 3.2 Standard notations and common functions. Distinguish logarithm bases, exponents, factorials, and iteration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Asymptotic notation

Choose a function and limit direction first. Here n is a natural number tending to infinity, f is eventually nonnegative, and the comparison function g is eventually positive. If g can vanish, use inequality definitions rather than division. A threshold can exclude finitely many small sizes from an asymptotic statement, but legal small program inputs still need correct handling.

O(g), Ω(g), and Θ(g) are **sets of functions**. The convention f=O(g) means membership, not numerical equality. The function may measure time, space, comparisons, or a mathematical quantity; the notation does not specify best, worst, or average case.

Insertion sort has worst-case $\Theta(n^2)$ and best-case $\Theta(n)$. A common $O(n^2)$ bound on all inputs does not give a common $\Omega(n^2)$ bound. Specify both the function and the input quantifiers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quantifiers and constants

Relation	Definition, holding for every integer $n \geq n_0$
$f \in O(g)$	some $c > 0, n_0$ give $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	some $c > 0, n_0$ give $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	some $c_1, c_2 > 0, n_0$ give $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

Constants may depend on the functions and a fixed algorithm, but not on n. Choosing c=n does not prove n^2=O(n). A threshold need not be smallest; any larger threshold works. Tight bounds require both sides, not a ratio limit or equal physical running times. Θ(g) is a subset of both O(g) and Ω(g). The true statement n=O(n^2) is loose; confusing O with Θ loses this distinction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-sided proof with explicit constants

For $f(n) = 3n^2 - 8n + 5$, at $n \geq 4$,

$$f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1.$$

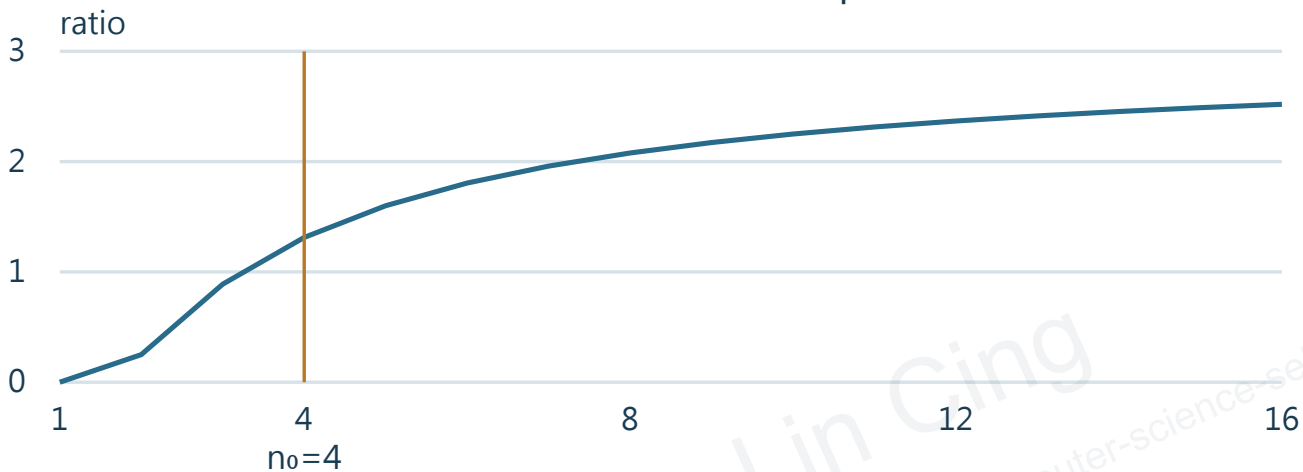
For $n \geq 1$, $-8n + 5 \leq 0$, so $f(n) \leq 3n^2$. Thus $c_1 = 1, c_2 = 3, n_0 = 4$ prove Θ(n²). At n=4, $16 \leq 21 \leq 48$; at n=8, $64 \leq 133 \leq 192$. The inequalities prove the entire tail; a finite table alone does not.

If f=O(n), some fixed c would give f≤cn eventually. Combining with f≥n² gives n≤c. An integer $n > \max(n_0, 4, c)$ contradicts this. A disproof must defeat **every candidate constant**, rather than one guessed constant.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A two-sided proof with explicit constants (continued)

Bounds must cover the entire tail, not selected points



Ratio $f(n)/n^2$

$f=3n^2-8n+5$

$n \geq 4$: $1 \leq \text{ratio} \leq 3$

$c_1=1$; $c_2=3$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Θ is the intersection of O and Ω

A Θ bound immediately supplies its upper constant for O and lower constant for Ω. Conversely, from $f \le c_U g$ after N_U and $f \ge c_L g$ after N_L, choose $n_0 = \max(N_U, N_L)$ to obtain both bounds simultaneously.

For eventually nonnegative f,g,

$$\max(f, g) \le f + g \le 2 \max(f, g),$$

so $f + g = \Theta(\max(f, g))$. Nonnegativity matters; f=n and g=−n cancel to zero.

For $p(n) = a_d n^d + \sum_{i<d} a_i n^i$, with $a_d > 0, d \ge 1$, put $M = \sum_{i<d} |a_i|$. At $n \ge \max(1, 2M/a_d)$, the lower-order absolute sum is at most $M n^{d-1} \le a_d n^d / 2$, giving

$$(a_d/2) n^d \le p(n) \le (3a_d/2) n^d.$$

For d=0, a positive constant is Θ(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Little-o and little-ω quantify over every constant

Membership $f \in o(g)$ means that **for every** c>0, a threshold $n_0(c)$ makes $0 \leq f(n) < cg(n)$ for all later n. Membership $f \in \omega(g)$ means that every c>0 eventually gives $0 \leq cg(n) < f(n)$. The threshold may depend on c; the chosen c may not depend on n.

For $7n=o(n^2)$, given c choose an integer threshold greater than 7/c. In contrast, $7n^2$ is not $o(n^2)$, since c=7 would require strict self-inequality. Also $n^2=\omega(7n)$, using n>7c. With positive denominators, little-o is equivalent to ratio limit zero, and little-ω to ratio limit infinity.

Although o(g) is contained in O(g), a loose O bound need not be little-o. A function equal to n on even n and n^2 on odd n belongs to $O(n^2)$, but neither $\Theta(n^2)$ nor $o(n^2)$: its ratio repeatedly returns to 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

What limits establish

For eventually positive g, if $L = \lim f/g$ exists, L=0 gives o(g), finite positive L gives Θ(g), and L=∞ with eventually positive f gives ω(g). For finite positive L, epsilon=L/2 gives $L/2 < f/g < 3L/2$ eventually, proving both bounds.

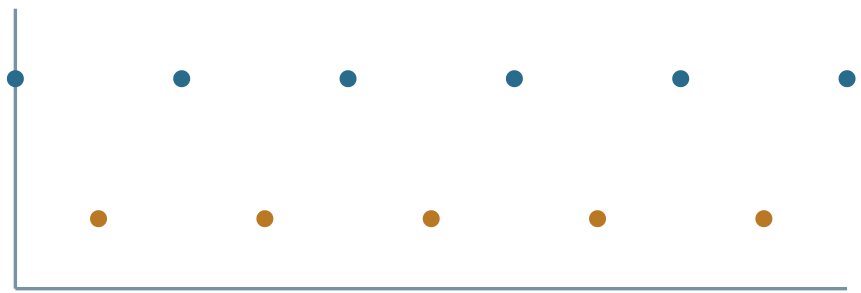
A nonexistent ratio limit need not mean incomparability. For $f = n(2 + (-1)^n), g = n$, the ratio alternates 1 and 3, yet $n \leq f \leq 3n$, giving Θ(n). A genuinely incomparable example has f=n² on even n and f=1 on odd n, with g=n. The even ratios n preclude O(g); the odd ratios 1/n preclude Ω(g). Each subsequence defeats every fixed candidate constant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

What limits establish (continued)

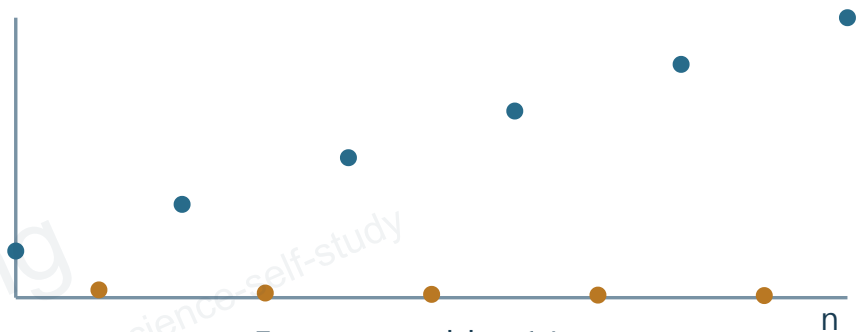
Plots of f/g: bounded between 1 and 3 on the left; separated subsequences on the right

Θ bounds; no ratio limit



Even n: 3; odd n: 1

Neither O nor Ω



Even n: n; odd n: 1/n

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algebraic properties

For eventually positive functions, O, Ω, and Θ are reflexive and transitive; Θ is symmetric. We have $f \in O(g)$ exactly when $g \in \Omega(f)$, and $f \in o(g)$ exactly when $g \in \omega(f)$. For example, $f \leq ag$ and $g \leq bh$ after their common threshold give $f \leq abh$, proving O transitivity. Reverse inequalities prove Ω transitivity; applying both proves Θ transitivity.

For little-o transitivity, given $c > 0$ use the first little-o bound with constant 1 and the second with c. Beyond the common threshold, $f < g < ch$. Inverting little-o relations proves little-ω transitivity. Neither strict relation is reflexive, and their intersection is empty: taking $c = 1$ in both would require $f < g$ and $g < f$ eventually.

These relations are not a total order. The parity example is neither O nor Ω relative to g, so arbitrary functions need not fall into smaller, equal-order, or larger categories.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Anonymous functions and one-way equalities

The expression $4n^2 + 6n + 2 = 4n^2 + \Theta(n)$ selects the particular member h=6n+2. The expression $4n^2 + \Theta(n) = \Theta(n^2)$ means that **every** left-hand choice of h yields a right-hand member. It does not assert equality of the sets or justify reversal.

Two O terms may select different anonymous functions. O(n)−O(n) does not guarantee Θ(n), since n−n=0. For signed errors, explicitly write $|e(n)| \leq cn$; this chapter's nonnegative O class does not automatically include negative functions.

If a single h satisfies h(i)=O(i), its constants work uniformly for all sufficiently large i. The finite prefix contributes a constant, so $\sum_{i=1}^{n} h(i) = O(n^2)$. Choosing a different constant depending on n for each term is invalid. The family $h_n(i) = ni$ can instead sum to Θ(n^3).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform bounds over inputs

Let $T(x) \geq 0$, $T_{max}(n) = \max_{|x|=n} T(x)$, and $T_{min}(n) = \min_{|x|=n} T(x)$, assuming nonempty input sets and attained extrema. A common Θ(g) bound means fixed c_L, c_U, n_0 such that **every** input x of every size n≥n₀ satisfies $c_L g(n) \leq T(x) \leq c_U g(n)$.

This is equivalent to $T_{max} \in O(g)$ and $T_{min} \in \Omega(g)$. Common bounds give the extrema bounds; conversely use $T_{min} \leq T(x) \leq T_{max}$ after a common threshold. A worst-case Θ(g) bound alone is insufficient because its lower bound applies only to some slow inputs.

For two parameters n,m, define the tail domain. Here it means both n≥n₀ **and** m≥m₀. A stronger domain where either is sufficiently large must explicitly use “or”; these quantifiers are not interchangeable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Standard notations and common functions

Nondecreasing means $m \leq n \Rightarrow f(m) \leq f(n)$; strictly increasing requires a strict conclusion for m<n. Decreasing notions reverse the inequalities. Nondecreasing does not imply strictly increasing.

For nondecreasing f,g, their sum is nondecreasing. Composition is too when g's range lies in f's domain. If both are nonnegative, $f(m)g(m) \leq f(n)g(m) \leq f(n)g(n)$, so their product is nondecreasing. Without nonnegativity the product claim can fail.

Floor is the greatest integer at most x; ceiling is the least integer at least x. Thus $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. For −2.3, floor is −3 and ceiling −2. Truncation toward zero is not general floor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integer splits, nested rounding, and mod

For integer n, $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$: even n=2k gives k+k; odd n=2k+1 gives (k+1)+k. For positive integers a,b and x≥0,

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor .$$

For integer k, $kb \le \lfloor x/a \rfloor$ is equivalent to $kab \le x$, so both sides choose the same greatest k. The ceiling identity similarly follows by choosing the least integer with reversed inequalities.

For positive integers a,b, $\lceil a/b \rceil \le (a+b-1)/b$ and $\lfloor a/b \rfloor \ge (a-b+1)/b$. Write a=qb+r, 0≤r<b, and separate r=0 from r>0.

For integer u and positive modulus m,

$$u \bmod m = u - m\lfloor u/m \rfloor \in \{0, \dots, m-1\}.$$

Thus −17 mod 5=−17−5(−4)=3, while 17 mod 5=2. A language remainder following the dividend's sign may differ. Congruence u≡v modulo m means m divides u−v.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Powers, exponentials, and fixed constants

For a>0 and real u,v, $a^{u+v} = a^u a^v$, $(a^u)^v = a^{uv}$, $a^0 = 1$, and $a^{-u} = 1/a^u$. A **fixed** a>1 gives increasing a^n, while 0<a<1 gives decreasing a^n. For n>0, n^b increases when b>0 and decreases when b<0.

Polynomially bounded means $O(n^k)$ for some fixed k, not necessarily a polynomial expression. The function $n^{\lg n}$ is not polynomially bounded, since for every fixed k its ratio to n^k is $n^{\lg n - k} \to \infty$. Conversely $2^{\lg n} = n$ is polynomial, while 2^n is exponential. A visible base 2 alone does not classify growth.

When a combinatorial formula needs 0^0, this chapter uses the empty-product value 1, not a universal value for all limits. Real-base identities cannot be applied unconditionally to negative bases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed-base exponentials dominate polynomials

Let a=1+δ>1 and b be fixed. For b≤0, $n^b/a^n \le 1/a^n \to 0$. For b>0, choose an integer k>b. The binomial expansion gives $a^n \ge \binom{n}{k}\delta^k$. At n≥2k, every numerator factor of the binomial coefficient is at least n/2, so

$$a^n \ge \frac{\delta^k n^k}{2^k k!}, \qquad 0 \le \frac{n^b}{a^n} \le \frac{2^k k!}{\delta^k} n^{b-k} \to 0.$$

Thus $n^b = o(a^n)$. The choice k depends on b, not on n.

For fixed ε>0 and fixed b, put t=lg n. Then $(\lg n)^b/n^\varepsilon = t^b/(2^\varepsilon)^t \to 0$. Every fixed polylogarithm is smaller than every positive power. A base approaching 1 with n violates the fixed-base premise; for example $(1+1/n)^n \to e$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Logarithm bases, powers, and composition

Use $\lg n = \log_2 n$, $\ln n = \log_e n$, $\lg^k n = (\lg n)^k$, $\lg \lg n = \lg(\lg n)$, and $\lg^{(k)} n$ for k iterations. A logarithm applies to its adjacent argument; lg n+q is not lg(n+q). Real log arguments must be positive, and bases positive and different from 1.

Taking log base c of $b^{\log_b x} = x$ gives $\log_b x = \log_c x / \log_c b$. Fixed b,c>1 differ by a positive constant, giving equal growth orders. Variable b=n gives $\log_n n = 1$, unlike lg n.

Products and powers obey $\log_b(xy) = \log_b x + \log_b y$ and $\log_b(x^r) = r \log_b x$. Also $a^{\log_b c} = c^{\log_b a}$, since both natural logarithms equal $(\ln a \ln c)/\ln b$. For example $3^{\lg 8} = 27 = 8^{\lg 3}$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bounds and errors for e and ln

The series is $e^x = \sum_{k \ge 0} x^k / k!$. Convexity gives $e^x \ge 1 + x$ for every real x; the tangent at zero has slope 1. Taylor's remainder gives $e^x = 1 + x + (e^\xi / 2) x^2$, with ξ between 0 and x. For $|x| \le 1$,

$$e^{-1} x^2 / 2 \le e^x - 1 - x \le e x^2 / 2,$$

proving a $\Theta(x^2)$ remainder as x tends to zero.

For $x > -1$, $x/(1+x) \le \ln(1+x) \le x$. For the upper bound, $h = x - \ln(1+x)$ has $h(0) = 0$ and derivative $x/(1+x)$, giving $h \ge 0$ on either side of zero. For the lower bound, $q = \ln(1+x) - x/(1+x)$ has derivative $x/(1+x)^2$ and the same argument applies. For $|x| < 1$,

$$\ln(1+x) = \sum_{k \ge 1} (-1)^{k+1} x^k / k.$$

Fix x and put $u = x/n$. Once $1 + u > 0$, the logarithmic bounds squeeze $n \ln(1 + x/n) \to x$. Exponentiation yields $(1 + x/n)^n \to e^x$. Domains and errors distinguish these limits from identities.

Factorial bounds from products

For integer $n \geq 0$, $0! = 1$ and $n! = n(n-1)!$. Every factor is at most n, so $n! \leq n^n$. The first $\lfloor n/2 \rfloor$ factors are at most $n/2$, giving

$$n!/n^n \leq 2^{-\lfloor n/2 \rfloor} \to 0,$$

and hence $n! = o(n^n)$. At least $\lfloor n/2 \rfloor$ final factors are at least $n/2$, so

$$n! \geq (n/2)^{\lfloor n/2 \rfloor}.$$

Taking logarithms bounds $\lg(n!)$ between $\lfloor n/2 \rfloor \lg(n/2)$ and $n \lg n$, proving $\Theta(n \log n)$. Subtracting $n \lg a$ from the lower bound tends to infinity for every fixed $a > 0$. Thus $n!/a^n \to \infty$, including $n! = \omega(2^n)$.

These proofs do not assume Stirling. Numerically, $6! = 720$, $2^6 = 64$, and $6^6 = 46656$; finite examples do not replace the general limits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling precision and constants in exponents

The standard Stirling estimate states, for n≥1,

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n}.$$

Thus the relative factor is $1 + \Theta(1/n)$, and

$$\lg(n!) = n\lg n - (\lg e)n + \tfrac{1}{2}\lg(2\pi n) + \Theta(1/n).$$

At n=6 the main term $\sqrt{12\pi}(6/e)^6$ is about 710.078; multiplying by factors between e^(1/73) and e^(1/72) brackets 720.

Fixed log bases differ by constants, but exponential bases cannot be ignored. We have $2^{n+3} = 8 \cdot 2^n = \Theta(2^n)$, while $2^{3n}/2^n = 4^n \to \infty$. If f=Θ(n), the safe conclusion is $2^f = 2^{\Theta(n)}$, not necessarily Θ(2ⁿ).

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Functional iteration and log-star stopping

Define $f^{(0)}(n) = n$ and $f^{(k+1)}(n) = f(f^{(k)}(n))$. For f(n)=3n, iteration gives $3^k n$, not $(3n)^k$.

For n>0,

$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

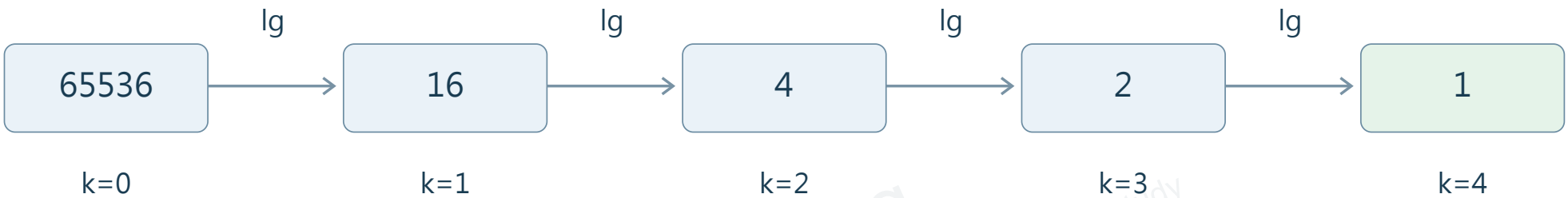
Take another logarithm only while the current value exceeds 1, avoiding nonpositive arguments. For $n \leq 1$, the answer is 0. The chain $65536 \to 16 \to 4 \to 2 \to 1$ gives log-star 4. Taking two logarithms gives 4, whereas $(\lg 65536)^2 = 256$.

Set tower thresholds $a_0 = 1$ and $a_{k+1} = 2^{a_k}$. For $n \in (a_{k-1}, a_k]$, log-star equals k. Thresholds for k=0 through 4 are 1,2,4,16,65536. Growth is slow but unbounded: for every finite k, some n exceeds a_k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Functional iteration and log-star stopping (continued)

Iterate lg once per arrow; stop at 1



Continue only above 1. Iteration 4 reaches 1, so lg*65536=4.

Two lg iterations give 4; $(\lg 65536)^2 = 256$. Iteration differs from a power.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving the Fibonacci closed form

Let $F_0=0$, $F_1=1$, and $F_n = F_{n-1} + F_{n-2}$. Both $\varphi = (1+\sqrt{5})/2$ and $\psi = (1-\sqrt{5})/2$ satisfy $z^2=z+1$. Define $G_n = (\varphi^n - \psi^n)/\sqrt{5}$. Its first values are 0 and 1. Both root-power sequences satisfy the same recurrence, so G does too. Induction proves F=G.

Since $|\psi|<1$,

$$|F_n - \varphi^n/\sqrt{5}| = |\psi|^n/\sqrt{5} < 1/2,$$

including n=0. Therefore $F_n = \lfloor \varphi^n/\sqrt{5} + 1/2 \rfloor$. At n=6, the terms are approximately 8.024922 and .024922, whose difference is 8.

For sufficiently large n, the error bound gives $\varphi^n/(2\sqrt{5}) \le F_n \le 2\varphi^n/\sqrt{5}$, proving $\Theta(\phi^n)$. A constant-size real expression is not a constant-time exact floating-point algorithm for arbitrary n. The output itself has $\Theta(n)$ bits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study